



SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

MINI PROJECT

LECTURER:

DR. ZURAINI BINTI ALI SHAH

GROUP 3 MEMBER:

LIM YU HAN (A23CS0241)

EVELYN GOH YUAN QI (A23CS0222)

ANIS SAFIYYA BINTI JANAI (A23CS0049)

PRAVINRAJ A/L SIVABATHI (A23CS0171)

Table of Content

1.1 Synopsis	3
1.2 Objective	4
2.1 System requirements	5
2.2 System design	6
3.0 Development activities and coding in C++	7
.....	8
.....	9
4.1 Task Distribution	14
4 .2 Log Sheet.....	15

1.1 Synopsis

The case study that we decided to develop is Budget hotel Booking System. The Budget Hotel Booking System is a software solution designed to manage hotel bookings and customer check-ins/check-outs efficiently with the help of the queue data structure. The system supports two major user roles which are customers and hotel staff. It models how real-world hotels work, introducing order and efficiency into the workflow.

At the very core, the system uses a queue-based system, following the First-In-First-Out (FIFO) principle in which customer bookings are processed in the order they were received. The customers can book a room, check their booking status, and request a cancellation, while the hotel staff processes the queue by allowing check-ins and cancellations.

The key components of the system include:

- Customer Management: Each booking is assigned a customer with details of the person's name, contact, check-in and check-out dates. Customer can choose to cancel their booking by entering their name.
- Hotel Staff Operations: The queue is managed by staff, a way to process bookings, and plan accordingly for smooth check-in.

By incorporating the queue data structure in this system, the following is achieved:

- Efficiency: The bookings are processed sequentially without causing any delay or miscommunication.
- Scalability: Huge volumes of bookings received from the customers during the peak season.
- Fairness: First come, first served unless tagged for priority processing.

This project illustrates the use of queues and priority queues in real-world applications that guarantee an orderly customer-oriented booking procedure in budget hotels.

The existing classes and its attribute:

1. Customer:

- CustomerID: Unique identifier for each customer <int>
- Name: Customer's name <string>
- ContactNumber: Customer's contact information <string>
- CheckInDate: Check-in date in the format YYYY-MM-DD <string>
- CheckOutDate: Check-out date in the format YYYY-MM-DD <string>
- TotalPayment: Total payment for the stay <double>

2. Booking:

- Bookings: Array to store Customer objects (array of Customer)
- Front: Index of the first booking in the queue <integer>
- Rear: Index of the last booking in the queue <integer>
- Size: Current number of bookings in the queue <integer>

3. Staff:

- No specific instance attributes
- Staff-related data (usernames and passwords) is stored in local arrays within the hotelStaffLogin method.

1.2 Objective

The objective of the project is to design a simple, efficient system for handling hotel bookings using a queue. The system will:

- Handle Customer Bookings: Handle reservations in the order they are made, using the FIFO principle. Customers can make bookings and cancel their bookings.
- Process Check-ins and Reject Bookings: Allow hotel staff to process check-ins and reject bookings with ease.
- Provide Equal Service: Ensure the serving of customers is in the order of requests.

The system shall enable easy and reliable management of hotel booking from both customers' and staff's points of view.

2.1 System requirements

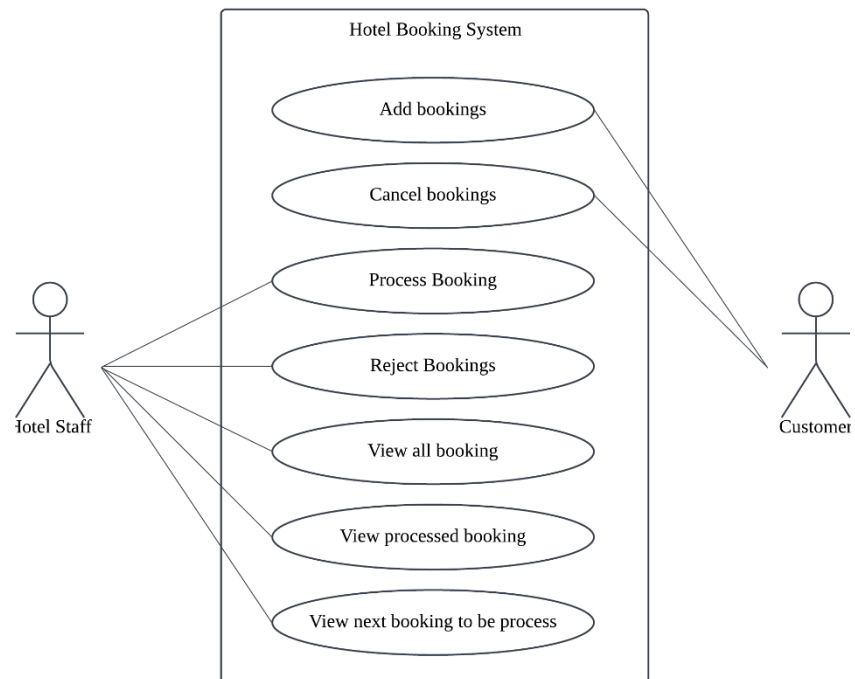


Figure 1 Use Case Diagram for Budget Hotel Booking System

2.2 System design

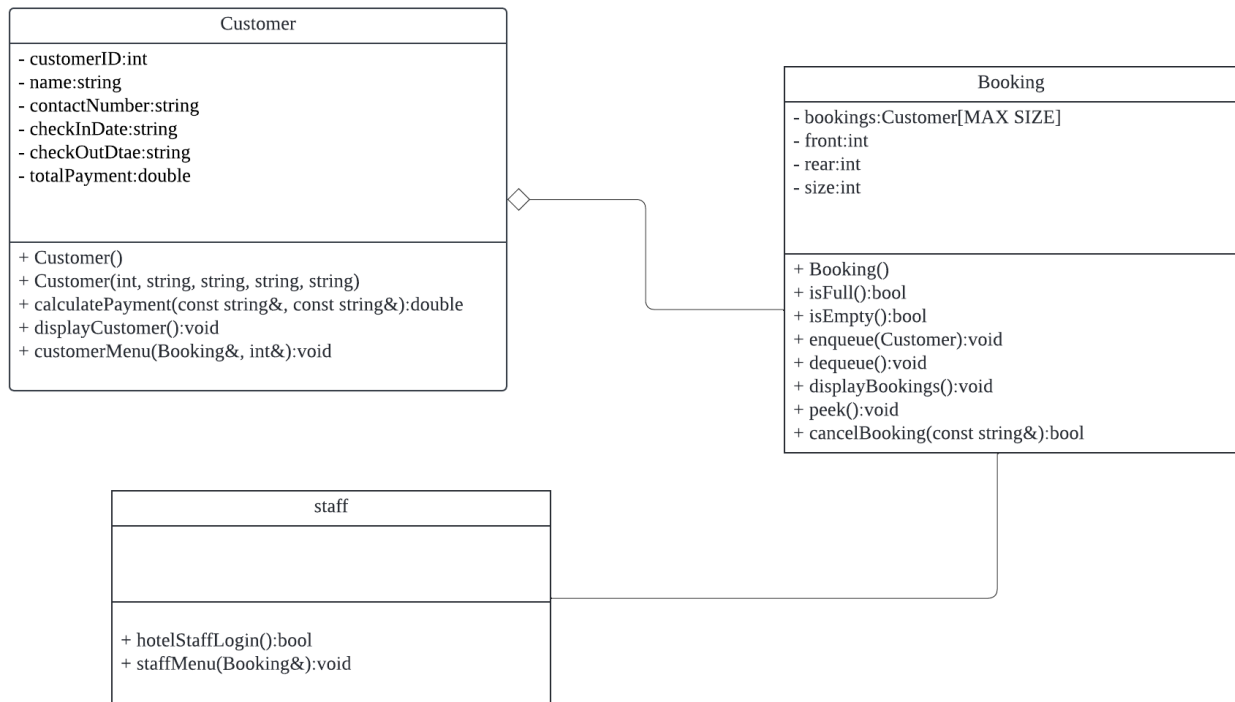


Figure 2 UML Class Diagram for Budget Hotel Booking System

3.0 Development activities and coding in C++

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
using namespace std;

#define MAX_SIZE 100

// Class to represent a customer
class Customer {
public:
    int customerID;
    string name;
    string contactNumber;
    string checkInDate;
    string checkOutDate;
    double totalPayment;

    Customer() {}

    Customer(int id, string n, string contact, string checkIn, string checkOut) {
        customerID = id;
        name = n;
        contactNumber = contact;
        checkInDate = checkIn;
        checkOutDate = checkOut;
        totalPayment = calculatePayment(checkIn, checkOut);
    }
};
```

```
// Simplified function to calculate the total payment based on check-in and check-out dates
double calculatePayment(const string& checkIn, const string& checkOut) {
    int checkInDay, checkOutDay;

    // Extracting days from the input dates (assuming YYYY-MM-DD format)
    checkInDay = stoi(checkIn.substr(8, 2));
    checkOutDay = stoi(checkOut.substr(8, 2));

    int days = checkOutDay - checkInDay;

    if (days <= 0) {
        cout << "Error: Check-out date must be after check-in date.\n";
        return 0;
    }

    return days * 99.00; // RM99 per night
}

void displayCustomer() {
    cout << "Customer ID: " << customerID << endl;
    cout << "Name: " << name << endl;
    cout << "Contact: " << contactNumber << endl;
    cout << "Check-In Date: " << checkInDate << endl;
    cout << "Check-Out Date: " << checkOutDate << endl;
    cout << "Total Payment: RM" << fixed << setprecision(2) << totalPayment << endl;
}

static void customerMenu(class Booking& hotelBooking, int& customerID);
};
```

```
// Queue Class
class Booking {
private:
    Customer bookings[MAX_SIZE];
    Customer processed[MAX_SIZE];
    int front;
    int rear;
    int size;
    int pSize; // processed bookings size

public:
    Booking() {
        front = 0;
        rear = -1;
        size = 0;
        pSize = 0;
    }
}
```

```
bool isFull() { return size == MAX_SIZE; }
bool isEmpty() { return size == 0; }

void enqueue(Customer c) {
    if (isFull()) {
        cout << "Queue is full. Cannot add more bookings.\n";
        return;
    }
    rear = (rear + 1) % MAX_SIZE;
    bookings[rear] = c;
    size++;
    cout << "Booking added successfully for " << c.name << ".\n";
}

void dequeue() {
    if (isEmpty()) {
        cout << "Queue is empty. No bookings to process.\n";
        return;
    }
    // Process the booking
    cout << "Processing booking for " << bookings[front].name << "... \n";
    addp(bookings[front]);
    front = (front + 1) % MAX_SIZE;
    size--;
}
}
```

```
// add processed bookings
void addp(Customer c) {
    if (pSize == MAX_SIZE) {
        cout << "Processed bookings storage is full.\n";
        return;
    }
    processed[pSize++] = c;
}

void displayBookings() {
    if (isEmpty()) {
        cout << "No bookings in the queue.\n";
        return;
    }

    cout << "\n=== Current Bookings ===\n";
    int count = 0;
    for (int i = front; count < size; i = (i + 1) % MAX_SIZE, count++) {
        cout << "Booking " << count + 1 << ":\n";
        bookings[i].displayCustomer();
        cout << "-----\n";
    }
}
}
```



```

// add processed bookings
void addp(Customer c) {
    if (pSize == MAX_SIZE) {
        cout << "Processed bookings storage is full.\n";
        return;
    }
    processed[pSize++] = c;
}

void displayBookings() {
    if (isEmpty()) {
        cout << "No bookings in the queue.\n";
        return;
    }

    cout << "\n=== Current Bookings ===\n";
    int count = 0;
    for (int i = front; count < size; i = (i + 1) % MAX_SIZE, count++) {
        cout << "Booking " << count + 1 << ":\n";
        bookings[i].displayCustomer();
        cout << "-----\n";
    }
}

```

```

void rejectBooking(const string& name) {
    if (isEmpty()) {
        cout << "No bookings to reject.\n";
        return;
    }

    int found = -1;

    for (int i = front; i != rear; i = (i + 1) % MAX_SIZE) {
        if (bookings[i].name == name) {
            found = i;
            break;
        }
    }

    if (bookings[rear].name == name) {
        found = rear;
    }

    if (found == -1) {
        cout << "No booking found with name: " << name << "\n";
        return;
    }

    for (int i = found; i != rear; i = (i + 1) % MAX_SIZE) {
        bookings[i] = bookings[(i + 1) % MAX_SIZE];
    }
    rear = (rear - 1 + MAX_SIZE) % MAX_SIZE;
    size--;
    cout << "Booking for " << name << " has been rejected by staff.\n";
}

```

```

void cancelBooking(const string& name) {
    if (isEmpty()) {
        cout << "No bookings to cancel.\n";
        return;
    }

    int found = -1;
    // Search for the booking
    for (int i = front; i != rear; i = (i + 1) % MAX_SIZE) {
        if (bookings[i].name == name) {
            found = i;
            break;
        }
    }
    // Check the rear position as well
    if (bookings[rear].name == name) {
        found = rear;
    }

    if (found == -1) {
        cout << "No booking found with name: " << name << "\n";
        return;
    }

    // Remove the booking and shift elements
    for (int i = found; i != rear; i = (i + 1) % MAX_SIZE) {
        bookings[i] = bookings[(i + 1) % MAX_SIZE];
    }
    rear = (rear - 1 + MAX_SIZE) % MAX_SIZE;
    size--;
    cout << "Booking for " << name << " has been cancelled.\n";
}

```

```

    case 2: {
        string name;
        cout << "Enter Customer Name to cancel booking: ";
        cin.ignore();
        getline(cin, name);
        hotelBooking.cancelBooking(name);
        break;
    }
    case 3:
        cout << "Exiting Customer Menu.\n";
        break;
    default:
        cout << "Invalid choice. Please try again.\n";
    }
} while (choice != 3);
}

```

```

class Staff {
public:
    // Hotel Staff login system
    static bool hotelStaffLogin() {
        const int numStaff = 2;
        string usernames[numStaff] = {"staff1", "staff2"};
        string passwords[numStaff] = {"password1", "password2"};

        string username, password;
        cout << "Enter username: ";
        cin >> username;
        cout << "Enter password: ";
        cin >> password;

        for (int i = 0; i < numStaff; ++i) {
            if (usernames[i] == username && passwords[i] == password) {
                cout << "Login successful. Welcome, " << username << "!\n";
                return true;
            }
        }
        cout << "Invalid credentials. Access denied.\n";
        return false;
    }

    // Staff menu
    static void staffMenu(Booking& hotelBooking) {
        int choice;
        do {
            cout << "\n=== Hotel Staff Menu ===\n";
            cout << "1. Process Booking\n";
            cout << "2. View Next Booking\n";
            cout << "3. View All Bookings\n";
            cout << "4. View Processed Bookings\n";
        } while (choice < 5);
    }
};

```

```

switch (choice) {
case 1:
    hotelBooking.dequeue();
    break;
case 2:
    hotelBooking.peak();
    break;
case 3:
    hotelBooking.displayBookings();
    break;
case 4:
    hotelBooking.displayp();
    break;
case 5: {
    string name;
    cout << "Enter Customer Name to reject booking: ";
    cin.ignore();
    getline(cin, name);
    hotelBooking.rejectBooking(name);
    break;
}
case 6:
    cout << "Logging out.\n";
    break;
default:
    cout << "Invalid choice. Please try again.\n";
}
} while (choice >= 1 && choice < 6);
}

```

```

int main() {
    Booking hotelBooking;
    int customerID = 1;
    int mainChoice;

    do {
        cout << "\n=== Budget Hotel Booking System ===\n";
        cout << "1. Customer\n";
        cout << "2. Hotel Staff\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> mainChoice;

        switch (mainChoice) {
            case 1:
                Customer::customerMenu(hotelBooking, customerID);
                break;
            case 2:
                if (Staff::hotelStaffLogin()) {
                    Staff::staffMenu(hotelBooking);
                }
                break;
            case 3:
                cout << "Exiting the system. Goodbye!\n";
                break;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    } while (mainChoice != 3);

    return 0;
}

```

Output Terminal :

```
=== Budget Hotel Booking System ===
1. Customer
2. Hotel Staff
3. Exit
Enter your choice: 1

=== Customer Menu ===
1. Add Booking
2. Cancel Booking
3. Exit
Enter your choice: 1
Enter your name: Pravinraj
Enter contact number: 0173808226
Enter check-in date (YYYY-MM-DD): 2025-01-10
Enter check-out date (YYYY-MM-DD): 2025-01-15
Booking added successfully for Pravinraj.

=== Customer Menu ===
1. Add Booking
2. Cancel Booking
3. Exit
Enter your choice: 3
Exiting Customer Menu.

=== Budget Hotel Booking System ===
1. Customer
2. Hotel Staff
3. Exit
Enter your choice: 2
Enter username: staff1
Enter password: password1
Login successful. Welcome, staff1!

=== Hotel Staff Menu ===
1. Process Booking
2. View Next Booking
3. View All Bookings
4. View Processed Bookings
5. Reject Bookings
6. Logout
Enter your choice: 1
Processing booking for Pravinraj...
```

```

=== Hotel Staff Menu ===
1. Process Booking
2. View Next Booking
3. View All Bookings
4. View Processed Bookings
5. Reject Bookings
6. Logout
Enter your choice: 4

=== Processed Bookings ===
Processed Booking 1:
Customer ID: 1
Name: Pravinraj
Contact: 0173808226
Check-In Date: 2025-01-10
Check-Out Date: 2025-01-15
Total Payment: RM495.00
-----
Total Processed Bookings: 1

=== Hotel Staff Menu ===
1. Process Booking
2. View Next Booking
3. View All Bookings
4. View Processed Bookings
5. Reject Bookings
6. Logout
Enter your choice: 5
Enter Customer Name to reject booking: pravinraj
No bookings to reject.

=== Hotel Staff Menu ===
1. Process Booking
2. View Next Booking
3. View All Bookings
4. View Processed Bookings
5. Reject Bookings
6. Logout
Enter your choice: 6
Logging out.

=== Budget Hotel Booking System ===
1. Customer
2. Hotel Staff
3. Exit
Enter your choice: █

```

4.1 Task Distribution

Task	Assigned To	Details
Requirement Analysis	Evelyn Lim	Gather system requirements.
System requirements (Use Case)	Anis	Design use case diagram for the system
System Design (Class Diagram)	Pravinraj	Create ADTs and design class relationships.
Coding Implementation	Evelyn Lim	Write the code for the system.
Testing	Anis	Test all functionalities for bugs.
Documentation	Pravinraj Anis	Compile system documentation

4 .2 Log Sheet

Date	Activity	Members Involved
02 / 01 / 2025	First discussion and selection of case study.	All members
09 / 01 / 2025	Prepare a clear synopsis and objective for the project, outline the system's purpose, goals and practical application of queue data structures for efficient booking management.	Evelyn Lim
09 / 01 / 2025	Analyzing and defining system requirements to make a clear use case diagram.	Anis
09 / 01 / 2025	UML class diagrams are created with clearly defined classes and attributes, representing a well-structured and organized system.	Pravinraj
10 / 01 / 2025	Develop functionalities including peek, displayBooking, and cancelBooking, ensuring smooth queue management and enhanced usability.	Lim
10 / 01 / 2025	Develop the Customer class, Staff class for login and staff's menu, and contributed to the key functions in the Booking class, including enqueue(), dequeue() and displayBookings().	Evelyn
10 / 01 / 2025	Develop the totalPayment function to return the exact booking payments due in conformance with days and helped to develop Staff class and main.	Pravinraj
11 / 01 / 2025	Developed the processed booking function that displays the processed bookings and counts the total number of processed bookings. Do system testing to identify and fix errors.	Anis
11 / 01 / 2025	Finalized the documentation	Pravinraj Anis

Video Demo Link

<https://youtu.be/7VdzJtP3Kgw>